

Chapter 7

Advanced SQL

1

**COMPANY
Database**

EMPLOYEE									
Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT			
Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS	
Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON		
Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT			
Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT				
Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

2

SELECT-FROM-WHERE

```
SELECT <attr and func list>
FROM <table list>
[ WHERE <condition> ]
[ ORDER BY <attr list> ];
```

EXAMPLES:

WHERE ... **AND** ... **OR**

WHERE Address **LIKE** '%Houston TX%';

WHERE Bdate **LIKE** '_ _ 6%';

WHERE (Salary **BETWEEN** 30000 **AND** 40000) **AND** Dno = 5;

3

Two-Valued Logic

AND	T	F
T	T	F
F	F	F

OR	T	F
T	T	T
F	F	F

NOT	
T	F
F	T

SQL uses a three-valued logic:

TRUE, **FALSE**, and **UNKNOWN** (like MAYBE)

Three-Valued Logic

AND	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

OR	T	F	U
T	T	T	T
F	F	F	U
U	U	U	U

NOT	
T	F
F	T
U	U

T	1
F	0
U	0.5

AND(x,y) = min(x,y)
OR(x,y) = max(x,y)
NOT(x) = 1-x

4

NULL and Three-Valued Logic

- Meanings of **NULL**
 - Unknown value
 - Unavailable or withheld value
 - Not applicable attribute
- Each individual **NULL** value considered to be different from every other **NULL** value

These are **UNKNOWN**:

```
NULL = 1  
NULL <> 1  
NULL > 1  
NULL = NULL
```

5

NULL

SQL can check whether an attribute value is **NULL**

IS NULL or **IS NOT NULL**

Query: Retrieve the names of all employees who do not have supervisors.

```
SELECT Lname, Fname  
FROM EMPLOYEE  
WHERE Super_ssn IS NULL;
```

6

Nested Queries

List all employees whose salary is greater than the average salary

Step 1: Find the average salary

```
SELECT AVG(Salary)
FROM EMPLOYEE
```

Step 2: List Ssn, Lname, Fname and Salary of all employees

```
SELECT Ssn, Lname, Fname, Salary
FROM EMPLOYEE
```

Step 3: Filter all employees on Salary > the average

```
SELECT Ssn, Lname, Fname, Salary
FROM EMPLOYEE
WHERE Salary > AVG
```

7

Nested Queries

List all employees whose salary is greater than the average salary

```
SELECT Ssn, Lname, Fname, Salary
FROM EMPLOYEE
WHERE Salary > (SELECT AVG(Salary) FROM EMPLOYEE) ;
```

> operator compares to
a single value

subquery should return a
single value (35125)

Ssn	Lname	Fname	Salary
333445555	Wong	Franklin	40000
987654321	Wallace	Jennifer	43000
666884444	Narayan	Ramesh	38000
888665555	Borg	James	55000

8

Nested Queries

List all project numbers for projects that involve an employee whose last name is 'Wong' as a worker.

```
SELECT    Pno
FROM      WORKS_ON, EMPLOYEE
WHERE     Essn=Ssn AND Lname='Wong';
```

Pno
2
3
10
20

9

Nested Queries

List all project numbers for projects that involve an employee whose last name is 'Wong' as a manager of the department that controls the project.

```
SELECT    Pnumber
FROM      PROJECT, DEPARTMENT, EMPLOYEE
WHERE     Dno=Dnumber AND Mgr_ssn=Ssn
AND Lname= 'Wong';
```

Pnumber
1
2
3
10
20
30

10

Nested Queries

List all project numbers for projects that involve an employee whose last name is 'Wong', either as a worker or as a manager of the department that controls the project.

Pnumber
2
3
10
20
1
30

```
SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN (SELECT Pno
                   FROM WORKS_ON, EMPLOYEE
                   WHERE Essn=Ssn AND Lname='Wong')
OR Pnumber IN (SELECT Pnumber
               FROM PROJECT, DEPARTMENT, EMPLOYEE
               WHERE Dno=Dnumber AND Mgr_ssn=Ssn AND Lname= 'Wong') ;
```

11

Nested Queries

Use tuples of values in comparison. Place them within parentheses

```
SELECT DISTINCT Essn
FROM WORKS_ON
WHERE (Pno, Hours) IN (SELECT Pno, Hours
                       FROM WORKS_ON
                       WHERE Essn='123456789') ;
```

Essn
123456789

12

Correlated Nested Queries

Avoid potential errors and ambiguities. Create tuple variables (aliases) for all tables referenced in the query

Query: Retrieve the name of each employee who has a dependent with the same sex (as the employee).

```
SELECT E.Fname, E.Lname
FROM   EMPLOYEE E
WHERE  E.Ssn IN (SELECT Essn
                FROM DEPENDENT D
                WHERE E.Sex=D.Sex) ;
```

Fname	Lname
John	Smith
Franklin	Wong

Correlated nested query is evaluated separately for each tuple in the outer query

13

Nested Queries

Queries that are nested using the = or IN comparison operator can be collapsed into a single block:

```
SELECT E.Fname, E.Lname
FROM   EMPLOYEE E
WHERE  E.Ssn IN (SELECT Essn
                FROM DEPENDENT D
                WHERE E.Sex=D.Sex) ;
```

Fname	Lname
John	Smith
Franklin	Wong



```
SELECT E.Fname, E.Lname
FROM   EMPLOYEE E, DEPENDENT D
WHERE  E.Ssn=D.Essn AND E.Sex=D.Sex ;
```

Fname	Lname
Franklin	Wong
John	Smith

14

EXISTS

EXISTS and NOT EXISTS

- Checks a table (or result of query) is empty or not. They are *Boolean functions* that return **TRUE** or **FALSE**.
- Typically used in conjunction with a correlated nested query

Query: List the names of managers who have at least one dependent.

```
SELECT Ssn, Fname, Lname
FROM EMPLOYEE
WHERE EXISTS ( SELECT * FROM DEPENDENT
                WHERE Ssn=Essn )
      AND EXISTS ( SELECT * FROM DEPARTMENT
                WHERE Ssn=Mgr_ssn );
```

Ssn	Fname	Lname
333445555	Franklin	Wong
987654321	Jennifer	Wallace

15

NOT EXISTS

Query: Retrieve the names of employees who have no dependents.

```
SELECT Ssn, Fname, Lname
FROM EMPLOYEE E
WHERE NOT EXISTS (SELECT *
                  FROM DEPENDENT D
                  WHERE E.Ssn = D.Essn);
```

Ssn	Fname	Lname
999887777	Alicia	Zalaya
666884444	Ramesh	Narayan
453453453	Joyce	English
987987987	Ahmad	Jabbar
888665555	James	Borg

16

NOT EXISTS

Query: List first and last name of employees who work on **ALL** projects controlled by department #4.

```
SELECT FNAME, LNAME
FROM EMPLOYEE E
WHERE NOT EXISTS
      (SELECT Pnumber
       FROM PROJECT P
       WHERE P.Dnum = 4
        EXCEPT
        SELECT Pno
        FROM WORKS_ON W
        WHERE W.Essn = E.Ssn);
```

Projects
controlled
by dept #4

Projects the
employee
works on

Fname	Lname
Alicia	Zalaya
Ahmad	Jabbar

Double negation: List those employees for whom there does NOT exist a project managed by department #4 that they do NOT work on.

17

Aggregate Functions

Summarizes information from multiple tuples into a single tuple

Built-in aggregate functions

COUNT, **SUM**, **MAX**, **MIN**, and **AVG**

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM EMPLOYEE;
```

SUM(Salary)	MAX(Salary)	MIN(Salary)	AVG(Salary)
281000	55000	25000	35125

18

Naming Results of Aggregation

```
SELECT      SUM(Salary) AS Total_Sal,  
            MAX(Salary) AS Highest_Sal,  
            MIN(Salary) AS Lowest_Sal,  
            AVG(Salary) AS Average_Sal  
FROM        EMPLOYEE;
```

Total_Sal	Highest_Sal	Lowest_Sal	Average_Sal
281000	55000	25000	35125

19

Aggregate Functions

NULL values are ignored by aggregate functions

Query: Find the number of employees of the Research department, as well as the total, maximum, minimum and average salary in this department.

```
SELECT COUNT(*)      AS Num_Emp,  
       SUM(Salary)   AS Total_Sal,  
       MAX(Salary)   AS Highest_Sal,  
       MIN(Salary)   AS Lowest_Sal,  
       AVG(Salary)   AS Average_Sal  
FROM   EMPLOYEE, DEPARTMENT  
WHERE  Dname='Research' AND Dno=Dnumber;
```

Num_Emp	Total_Sal	Highest_Sal	Lowest_Sal	Average_Sal
4	133000	40000	25000	33250

20

Expanded SELECT Queries

```
SELECT <attr and func list>  
FROM <table list>  
[ WHERE <condition> ]  
[ GROUP BY <grouping attr> ]  
[ HAVING <group cond> ]  
[ ORDER BY <attr list> ];
```

21

GROUP BY

Query: For each department, retrieve the department number, the number of employees in the department, and their average salary.

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

```
SELECT  Dno, COUNT(*), AVG(Salary)  
FROM    EMPLOYEE  
GROUP BY Dno;
```

22

GROUP BY

Query: For each project, retrieve the project number, the project name, and the number of employees who work on that project.

```
SELECT  Pnumber, Pname, COUNT(*) AS Nemp
FROM    PROJECT, WORKS_ON
WHERE   Pnumber=Pno
GROUP BY Pnumber;
```

Pnumber	Pname	Nemp
1	ProductX	2
2	ProductY	3
3	ProductZ	2
10	Computerization	3
20	Reorganization	3
30	Newbenefits	3

23

GROUP BY and HAVING

The **HAVING** clause provides a condition to filter entire groups.

Query: For each project **on which more than two employees work**, retrieve the project number, the project name, and the number of employees who work on the project.

```
SELECT  Pnumber, Pname, COUNT(*) AS Nemp
FROM    PROJECT, WORKS_ON
WHERE   Pnumber=Pno
GROUP BY Pnumber
HAVING  COUNT(*) > 2;
```

Pnumber	Pname	Nemp
2	ProductY	3
10	Computerization	3
20	Reorganization	3
30	Newbenefits	3

24

Combining WHERE and HAVING

Query: Count the **total** number of employees whose salaries exceed \$35,000 in each department employing three or more employees.

Incorrect

```
SELECT Dno, COUNT(*) AS Nemp
FROM EMPLOYEE
WHERE Salary>35000
GROUP BY Dno
HAVING COUNT(*)>=3;
```

Dno	Nemp
4	1
5	2

Correct

```
SELECT Dno, COUNT(*) AS Nemp
FROM EMPLOYEE
WHERE Salary>35000 AND Dno IN
      (SELECT Dno FROM EMPLOYEE
       GROUP BY Dno
        HAVING COUNT(*)>=3)
GROUP BY Dno;
```

Dno	Nemp
4	1
5	2

The **WHERE** clause applies tuple by tuple whereas **HAVING** applies to entire group of tuples

25

WITH

The **WITH** clause allows a user to define a table that will only be used in a particular query

Used for convenience to create a temporary “view” and use immediately in a query

Allows a more straightforward way of looking at a step-by-step query

QUERY: List the SSN, last name and salary of all employees earning more than the average salary.

```
WITH TEMP(av_sal) as
      (SELECT AVG(Salary)
       FROM EMPLOYEE)
SELECT Ssn, Lname, Salary
FROM EMPLOYEE E, TEMP T
WHERE E.Salary > T.av_sal;
```

Ssn	Lname	Salary
333445555	Wong	40000
987654321	Wallace	43000
666884444	Narayan	38000
888665555	Borg	55000

26

WITH

Query: Count the **total** number of employees whose salaries exceed \$35,000 in each department employing three or more employees.

Correct

```
SELECT Dno, COUNT(*)
FROM EMPLOYEE
WHERE Salary>35000 AND Dno IN
      (SELECT Dno FROM EMPLOYEE
       GROUP BY Dno
        HAVING COUNT(*)>=3)
GROUP BY Dno;
```

Also correct

```
WITH BIGDEPS(Dno2) AS
      (SELECT Dno FROM EMPLOYEE
       GROUP BY Dno
        HAVING COUNT(*)>=3)
SELECT Dno, COUNT(*)
FROM EMPLOYEE, BIGDEPS
WHERE Salary>35000 AND Dno=Dno2
GROUP BY Dno;
```

Dno	Nemp
4	1
5	2

27

CASE

A conditional expression

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  WHEN conditionN THEN resultN
  ELSE result
END;
```

28

CASE

Update: Give employees raises depending on their department.

```
UPDATE EMPLOYEE
SET   Salary =
      CASE
        WHEN Dno = 5 THEN Salary + 2000
        WHEN Dno = 4 THEN Salary + 1500
        WHEN Dno = 1 THEN Salary + 3000
        ELSE Salary + 0
      END
```

29

CASE

Compute the pension benefit for each employee based on their salary and these three brackets:

	Salary <\$30,000	Salary >\$30,000 and <\$50,000	Salary >\$50,000
BR1 <30K	60%	50% (\$15K)	40% (\$12K)
BR2 30K-50K		30%	20% (\$4K)
BR3 >\$50K			10%

30

CASE

```
SELECT Ssn, Fname, Lname, Salary,
CASE
  WHEN Salary<=30000 THEN Salary*0.6
  WHEN Salary>30000 AND Salary <= 50000 THEN 15000
  WHEN Salary>50000 THEN 12000
  ELSE 0
END AS PL1,
CASE
  WHEN Salary > 30000 AND Salary <= 50000 THEN ((Salary-30000)*0.30)
  WHEN Salary > 50000 THEN 4000
  ELSE 0
END AS PL2,
CASE
  WHEN Salary > 50000 THEN ((Salary-50000)*0.10)
  ELSE 0
END AS PL3,
CASE
  WHEN Salary<=30000 THEN Salary*0.6
  WHEN Salary>30000 AND Salary <= 50000 THEN 15000 + ((Salary-30000)*0.3)
  WHEN Salary>50000 THEN 16000+((Salary-50000)*0.10)
  ELSE 0
END AS TotalPension
FROM EMPLOYEE
ORDER BY Salary;
```

31

Ssn	Fname	Lname	Salary	PL1	PL2	PL3	TotalPension
999887777	Alicia	Zalaya	25000	15000	0	0	15000
453453453	Joyce	English	25000	15000	0	0	15000
987987987	Ahmad	Jabbar	25000	15000	0	0	15000
123456789	John	Smith	30000	18000	0	0	18000
666884444	Ramesh	Narayan	38000	15000	2400	0	17400
333445555	Franklin	Wong	40000	15000	3000	0	18000
987654321	Jennifer	Wallace	43000	15000	3900	0	18900
888665555	James	Borg	55000	12000	4000	500	16500

32

TRIGGER

Example: Fire a trigger if the salary of an employee exceeds that of its supervisor, when modifying the salary or the supervisor of an employee.

<pre>CREATE TRIGGER SALARY_VIOLATION BEFORE UPDATE OF Salary, Super_ssn ON EMPLOYEE</pre>	Event(s)
<pre>FOR EACH ROW WHEN (NEW.Salary > (SELECT Salary FROM EMPLOYEE WHERE Ssn=NEW.Super_ssn))</pre>	Condition
<pre>INFORM_SUPERVISOR(NEW.Super_ssn, NEW.Ssn);</pre>	Action

33

Triggers

The **CREATE TRIGGER** statement is used to monitor the database.

A trigger is a procedure automatically executed when one of the following events occur:

- A **database definition (DDL)** statement (**CREATE**, **ALTER**, or **DROP**)
- A **database manipulation (DML)** statement (**DELETE**, **INSERT**, or **UPDATE**)
- A **database operation** (**SERVERERROR**, **LOGON**, **LOGOFF**, **STARTUP**, or **SHUTDOWN**)

Triggers can be defined on the table, view, schema, or database with which the event is associated.

DROP TRIGGER cancels a trigger.

34

CREATE VIEW (virtual table)

```
CREATE VIEW WORKS_ON1 AS
  SELECT Fname, Lname, Pname, Hours
  FROM EMPLOYEE, PROJECT, WORKS_ON
  WHERE Ssn=Essn AND Pno=Pnumber;
```

WORKS_ON1

Fname	Lname	Pname	Hours
John	Smith	ProductX	32.5
John	Smith	ProductY	7.5
Ramesh	Narayan	ProductZ	40
Joyce	English	ProductX	20
Joyce	English	ProductY	20
Franklin	Wong	ProductY	10
Franklin	Wong	ProductZ	10
Franklin	Wong	Computerization	10
Franklin	Wong	Reorganization	10
Alicia	Zalaya	Newbenefits	30
Alicia	Zalaya	Computerization	10
Ahmad	Jabbar	Computerization	35
Ahmad	Jabbar	Newbenefits	5
Jennifer	Wallace	Newbenefits	20
Jennifer	Wallace	Reorganization	15
James	Borg	Reorganization	NULL

```
CREATE VIEW DEPT_INFO(Dept_name, No_of_emps, Total_sal) AS
  SELECT Dname, COUNT (*), SUM (Salary)
  FROM DEPARTMENT, EMPLOYEE
  WHERE Dnumber=Dno
  GROUP BY Dname;
```

DEPT_INFO

Dept_name	No_of_emps	Total_sal
Administration	3	93000
Headquarters	1	55000
Research	4	133000

35

Views in SQL

A view is:

- A single table derived from other tables called the **defining tables**
- Considered a virtual table that is not necessarily populated

Once a view is defined, SQL queries can use the view relation in the **FROM** clause

View is always up-to-date. This is the responsibility of the DBMS

DROP VIEW disposes of a view

36

Views as Authorization Mechanism

Views can be used to hide certain attributes or tuples from unauthorized users

E.g., A view for a user who is allowed to see only employee and project information for department #5:

```
CREATE VIEW DEPT5EMP AS
SELECT Fname, Lname, Pname, Hours
FROM   EMPLOYEE, PROJECT, WORKS_ON
WHERE  Ssn=Essn AND Pno=Pnumber AND Dno=5;
```

DEPT5EMP

Fname	Lname	Pname	Hours
John	Smith	ProductX	32.5
John	Smith	ProductY	7.5
Ramesh	Narayan	ProductZ	40
Joyce	English	ProductX	20
Joyce	English	ProductY	20
Franklin	Wong	ProductY	10
Franklin	Wong	ProductZ	10
Franklin	Wong	Computerization	10
Franklin	Wong	Reorganization	10

37

ALTER TABLE

Alter table actions include:

- Adding or dropping a column (attribute)
- Changing a column definition

Examples:

Add a new column to the **EMPLOYEE** table:

```
ALTER TABLE EMPLOYEE ADD COLUMN Job VARCHAR(12);
```

Remove a column from the **EMPLOYEE** table:

```
ALTER TABLE EMPLOYEE DROP COLUMN Address;
```

38

DROP

DROP command

- Used to drop named schema elements, such as tables, domains, or constraints

Drop behavior options:

- **CASCADE** and **RESTRICT**

Example:

```
DROP SCHEMA COMPANY CASCADE ;
```

Removes the schema and all its elements including tables, views, constraints, etc.